Music Streaming 10-Day Churn Prediction

Ziqi Meng Licheng Xiong

December 19, 2025

Agenda

1. Problem statement
2. Data overview
3. Sliding-window labeling
4. Feature engineering
5. Heavy-tail handling
6. Models
7. Ensemble: soft voting
8. Decision rule: Top-50%
9. Results

1. Problem statement

Goal: predict whether a user will **churn within 10 days** after an observation cutoff date c.

$$y(u, c) = \mathbf{1}\{\exists \text{ event of user } u \text{ with page}=\text{“Cancellation Confirmation”} \text{ in } (c, c + 10\text{d}]\}$$

- Test set: users observed up to 2018-11-20; need to predict churn in (2018-11-20, 2018-11-30].
- Key requirement: features must be computed using only events **up to cutoff**.

Takeaway: This is closer to a **time-to-event** problem than a static classifier.

2. Data overview

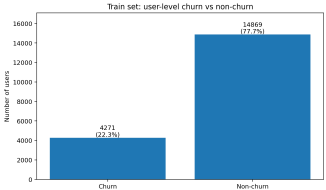
Event-level user logs (each row = one user event):

- Time: `ts`, `registration`, `time`
- Category: `page`, `level`, `status`, `gender`, `state`
- State extraction: `state = last 2 chars of location` (US state abbreviation).

Class balance mismatch

- In the training set, the class distribution is approximately 22% churn vs. 78% non-churn.
- The test set may have a different class proportion close to 50%:50%.
- Implication: focus on stable **ranking** and use a robust decision rule at submission time.

Training set distribution



3. Sliding-window labeling

Why we don't choose churn-anchored cutoffs?

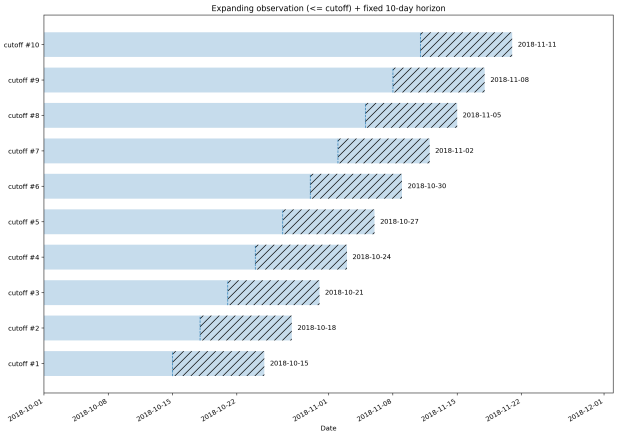
- Churn events occur at different times, so anchoring the cutoff on churn would use future information to define the observation window.
- This breaks the 10-day prediction horizon and introduces label leakage (cutoff depends on the outcome).
- Direct labeling (ever churn) $\rightarrow$ violates 10-day horizon + leakage.

Why sliding cutoffs?

- Single cutoff $\rightarrow$ wastes churn information before cutoff.
- **Sliding cutoffs** $\rightarrow$ mimics test-time structure and uses more data.

Takeaway: We train on **multiple versions of the past** so the model learns time-invariant signals.

How we construct training samples



Cutoffs start at cutoff_start; observation uses ALL history from obs_start.
Observation window is expanding: ts <= cutoff_date.

For each cutoff_date (moves forward 1 day):

- Observation window: all events with `ts` $\leq$ cutoff
- Prediction window: (c, c + 10 d]
- Keep only users "alive" at cutoff (no churn before cutoff)
- target $= 1$ if `Cancellation Confirmation` occurs in prediction window, else 0
- Each sample is (*userId*, *cutoff_date*)

4. Feature engineering (overview)

Category	Features
Static	gender, state
Lifecycle	days_since_registration
Recency & activity	n_events, recency_hours, active_days, total_listen_time
Rolling windows	events_last_[1/3/7]d, songs_last_[1/3/7]d
Session behavior	session_count; duration stats (mean/max/min/std); events-per-session stats
Page interaction	cnt_page_*, ratio_page_* (train/test unified vocab; per-cutoff)
Subscription	level_at_cutoff; ever_paid; n_level_change (per-cutoff)
Tech status	n_status_200,404,307, frac_status_404,200

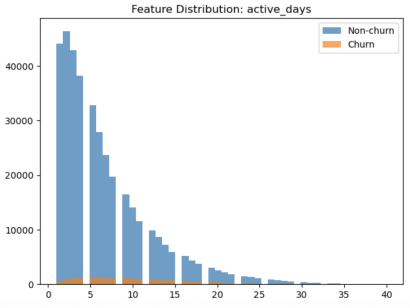
5. Heavy-tail handling: log1p + ratio normalization

- **Problem:** Count/duration features are strongly right-skewed (heavy-tailed).
- **Solution:** Apply

$$x \leftarrow \log(1 + x)$$

to counts/durations (e.g., `active_days`) to reduce skewness.

Raw distribution of `active_days`



6. Models

We train multiple base models on the engineered feature space:

- **LightGBM/Random Forest**: captures non-linear interactions, robust in high-dimensional sparse features.
- **Logistic Regression**: stable global ranking; we oversample minority class for stability.

Note on imbalance:

- Sliding windows amplify negatives (many non-churn cutoffs per user, finally 5% : 95%).
- Handle via: oversampling.

7. Ensemble: soft voting

We combine heterogeneous learners via probability averaging:

$$\hat{p}_{\text{vote}}(u) = \frac{w_1 \hat{p}_1(u) + w_2 \hat{p}_2(u) + \cdots + w_k \hat{p}_k(u)}{w_1 + w_2 + \cdots + w_k}$$

LR + LGBM + RF, where weights are tunable.

Motivation:

- LR: well-calibrated, stable
- Tree models: capture non-linear patterns induced by behavior sequences

8. Decision rule: Top-50%

Final submission converts probabilities to labels using a Top-50% rule:

$$\hat{y}(u) = \mathbf{1}\left\{\hat{p}(u) \geq q_{0.5}(\{\hat{p}(v)\})\right\}$$

where $q_{0.5}$ is the median of predicted probabilities in test users.

This matches the expected class balance assumption used in our submission scripts.

9. Results

According to the project report:

- Public leaderboard score: **0.692** (ranked **3rd**)
- Private leaderboard score: **0.642** (ranked **5th**)

Key drivers:

- Leakage-safe sliding-window construction
- Complementary ensemble (LR + tree-based)